```
!pip install torch torchvision
```

```
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.9.0+cpu)
Requirement already satisfied: torchvision in /usr/local/lib/python3.12/dist-packages (0.24.0+cpu)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch) (3.21.2)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from torchvision) (2.0.2)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /usr/local/lib/python3.12/dist-packages (from torchvision) (11.3
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch) (3.0.3)
```

```
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
import re
from collections import Counter
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from torch.utils.data import DataLoader, TensorDataset
```

```
df = pd.read_csv("/content/SMSSpamCollection (1)", sep='\t', header=None)
df.columns = ['label', 'text']

df['label'] = df['label'].map({'ham':0, 'spam':1})

print("Total samples:", len(df))
df.head()
```

```
Total samples: 5572
```

	label	text
0	0	Go until jurong point, crazy.. Available only ...
1	0	Ok lar... Joking wif u oni...
2	1	Free entry in 2 a wkly comp to win FA Cup fina...
3	0	U dun say so early hor... U c already then say...
4	0	Nah I don't think he goes to usf, he lives aro...

Next steps: Generate code with df New interactive sheet

```
def clean_text(text):
    text = text.lower()
    text = re.sub(r'[^a-zA-Z\s]', '', text)
    return text

df['text'] = df['text'].apply(clean_text)
df['tokens'] = df['text'].apply(lambda x: x.split())
```

```
all_words = [word for tokens in df['tokens'] for word in tokens]
word_counts = Counter(all_words)

vocab = {word:i+1 for i,(word,count) in enumerate(word_counts.items())}
vocab_size = len(vocab) + 1
```

```
print("Vocab size:", vocab_size)
```

```
Vocab size: 8630
```

```
!wget http://nlp.stanford.edu/data/glove.6B.zip
!unzip glove.6B.zip
```

```
--2026-02-19 04:41:41--  http://nlp.stanford.edu/data/glove.6B.zip
Resolving nlp.stanford.edu (nlp.stanford.edu)... 171.64.67.140
Connecting to nlp.stanford.edu (nlp.stanford.edu)|171.64.67.140|:80... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://nlp.stanford.edu/data/glove.6B.zip [following]
--2026-02-19 04:41:41--  https://nlp.stanford.edu/data/glove.6B.zip
Connecting to nlp.stanford.edu (nlp.stanford.edu)|171.64.67.140|:443... connected.
HTTP request sent, awaiting response... 301 Moved Permanently
Location: https://downloads.cs.stanford.edu/nlp/data/glove.6B.zip [following]
--2026-02-19 04:41:42--  https://downloads.cs.stanford.edu/nlp/data/glove.6B.zip
Resolving downloads.cs.stanford.edu (downloads.cs.stanford.edu)... 171.64.64.22
Connecting to downloads.cs.stanford.edu (downloads.cs.stanford.edu)|171.64.64.22|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 862182613 (822M) [application/zip]
Saving to: ‘glove.6B.zip’

glove.6B.zip        100%[===================>] 822.24M  2.85MB/s    in 3m 30s

2026-02-19 04:45:14 (3.91 MB/s) - ‘glove.6B.zip’ saved [862182613/862182613]

Archive:  glove.6B.zip
  inflating: glove.6B.50d.txt
  inflating: glove.6B.100d.txt
  inflating: glove.6B.200d.txt
  inflating: glove.6B.300d.txt
```

```
embedding_dim = 100
embeddings_index = {}

with open("glove.6B.100d.txt", encoding='utf8') as f:
    for line in f:
        values = line.split()
        word = values[0]
        vector = np.asarray(values[1:], dtype='float32')
        embeddings_index[word] = vector

embedding_matrix = np.zeros((vocab_size, embedding_dim))

for word, i in vocab.items():
    vector = embeddings_index.get(word)
    if vector is not None:
        embedding_matrix[i] = vector
```

```
max_len = 50

def text_to_sequence(tokens):
    seq = [vocab[word] for word in tokens if word in vocab]
    if len(seq) < max_len:
        seq += [0]*(max_len - len(seq))
    else:
        seq = seq[:max_len]
    return seq

X = np.array([text_to_sequence(tokens) for tokens in df['tokens']])
y = np.array(df['label'])
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

X_train = torch.LongTensor(X_train)
X_test = torch.LongTensor(X_test)
```

```
y_train = torch.FloatTensor(y_train)
y_test = torch.FloatTensor(y_test)
```

```
batch_size = 64

train_dataset = TensorDataset(X_train, y_train)
test_dataset = TensorDataset(X_test, y_test)

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=batch_size)
```

```
class TextCNN(nn.Module):
    def __init__(self, vocab_size, embedding_dim, embedding_matrix):
        super(TextCNN, self).__init__()

        self.embedding = nn.Embedding(vocab_size, embedding_dim)
        self.embedding.weight = nn.Parameter(torch.tensor(embedding_matrix, dtype=torch.float32))
        self.embedding.weight.requires_grad = False

        self.conv1 = nn.Conv1d(embedding_dim, 128, kernel_size=3)
        self.conv2 = nn.Conv1d(128, 64, kernel_size=3)

        self.relu = nn.ReLU()
        self.pool = nn.MaxPool1d(2)
        self.dropout = nn.Dropout(0.5)

        self.fc = nn.Linear(64*11, 1)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        x = self.embedding(x)
        x = x.permute(0,2,1)

        x = self.relu(self.conv1(x))
        x = self.pool(x)

        x = self.relu(self.conv2(x))
        x = self.pool(x)

        x = x.view(x.size(0), -1)
        x = self.dropout(x)
        x = self.fc(x)

        return self.sigmoid(x)
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

model = TextCNN(vocab_size, embedding_dim, embedding_matrix).to(device)

criterion = nn.BCELoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

epochs = 10

for epoch in range(epochs):
    model.train()
    total_loss = 0

    for inputs, labels in train_loader:
        inputs, labels = inputs.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(inputs).squeeze()
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        total_loss += loss.item()
```

```
    print(f"Epoch {epoch+1}, Loss: {total_loss/len(train_loader)}")
```

```
Epoch 1, Loss: 0.3478968679904938
Epoch 2, Loss: 0.15077604874968528
Epoch 3, Loss: 0.09877236187458038
Epoch 4, Loss: 0.08226975370198489
Epoch 5, Loss: 0.06574138892548424
Epoch 6, Loss: 0.055334994961906754
Epoch 7, Loss: 0.036730912933126095
Epoch 8, Loss: 0.031372027581424586
Epoch 9, Loss: 0.02873290628527424
Epoch 10, Loss: 0.02078473820417587
```

```
model.eval()
all_preds = []
all_labels = []

with torch.no_grad():
    for inputs, labels in test_loader:
        inputs = inputs.to(device)
        outputs = model(inputs).squeeze()
        preds = (outputs > 0.5).int().cpu()

        all_preds.extend(preds.numpy())
        all_labels.extend(labels.numpy())

print("Accuracy:", accuracy_score(all_labels, all_preds))
print(classification_report(all_labels, all_preds))
print("Confusion Matrix:\n", confusion_matrix(all_labels, all_preds))
```

```
Accuracy: 0.9766816143497757
              precision    recall  f1-score   support

         0.0       0.98      0.99      0.99       966
         1.0       0.93      0.89      0.91       149

    accuracy                           0.98      1115
   macro avg       0.96      0.94      0.95      1115
weighted avg       0.98      0.98      0.98      1115

Confusion Matrix:
 [[956  10]
 [ 16 133]]
```